

Matriz de Casos de Prueba — Bustoke Admin Backend

Proyecto: Bustoke Admin API (FastAPI + async SQLAlchemy 2.x + PostgreSQL 15)
Estándares: ISTQB / ISO/IEC 29119
Enfoques: Caja Negra (Partición de Equivalencia + VLC), Caja Blanca (Cobertura de Caminos), Basada en Riesgos

1. Pruebas de Caja Negra

1.1 Módulo Auth — Login (POST /admin/auth/login)

ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	Datos de Entrada
CN-01	Auth	Caja Negra — PE	Login exitoso con credenciales válidas de superadmin	Usuario existe en BD con email admin@bustoke.pe , rol superadmin , activo = true	{ "email": "admin@bustoke.pe", "password": "Admin123!" }
CN-02	Auth	Caja Negra — PE	Login con email no registrado	No existe usuario con ese email en tabla usuarios	{ "email": "noexiste@test.com", "password": "Cualquier123!" }
CN-03	Auth	Caja Negra — PE	Login con contraseña incorrecta	Usuario existe con email user@bustoke.pe , pero se envía password distinto a su hash	{ "email": "user@bustoke.pe", "password": "WrongPass999!" }
CN-04	Auth	Caja Negra — PE	Login de usuario inactivo	Usuario existe con email inactivo@bustoke.pe , activo = false	{ "email": "inactivo@bustoke.pe", "password": "SuPassword123" }
CN-05	Auth	Caja Negra — VLC	Email con formato inválido (partición inválida)	—	{ "email": "correo-invalido", "password": "Test123!" }
CN-06	Auth	Caja Negra — PE	Login con email en mayúsculas (case insensitive)	Usuario registrado con email test@bustoke.pe	{ "email": "TEST@BUSTOKE.PE", "password": "Password1" }

1.2 Módulo Auth — Refresh Token (POST /admin/auth/refresh)

ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	Datos de Entrada	Pasos de Ejec
CN-07	Auth	Caja Negra	Refresh exitoso con	Login previo exitoso que	{ "refreshToken": "<refresh_token_jwt>" }	1. POST /admin/auth

ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	Datos de Entrada	Pasos de Ejec
		— PE	refresh token válido	devuelve refreshToken válido		con body.
CN-08	Auth	Caja Negra — PE	Refresh con token de tipo access (invalido)	Obtener un accessToken vía login	{ "refreshToken": "<access_token_jwt>" }	1. POST /admin/auth con body.
CN-09	Auth	Caja Negra — VLC	Refresh con token expirado	Token refresh con exp en el pasado (modificar manualmente o esperar 7 días)	{ "refreshToken": "<refresh_token_expirado>" }	1. POST /admin/auth con body.
CN-10	Auth	Caja Negra — PE	Refresh con token malformado (string aleatorio)	—	{ "refreshToken": "abc123.token.invalido" }	1. POST /admin/auth con body.

1.3 Módulo Viajes — CRUD (/admin/viajes)

ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	
CN-11	Viajes	Caja Negra — VLC	Listar viajes con límite en borde inferior (limit=1)	Existen ≥ 2 viajes en BD	GET /admin/viajes?skip=0&limit=1 con token AdminOrSuper
CN-12	Viajes	Caja Negra — VLC	Listar viajes con límite en borde superior (limit=500)	Existen ≥ 500 viajes en BD	GET /admin/viajes?skip=0&limit=500 con token AdminOrSuper
CN-13	Viajes	Caja Negra — VLC	Listar viajes con límite sobre el máximo (limit=501)	—	GET /admin/viajes?skip=0&limit=501 con token AdminOrSuper
CN-14	Viajes	Caja Negra — VLC	Listar viajes con skip negativo (skip=-1)	—	GET /admin/viajes?skip=-1&limit=10 con token AdminOrSuper
CN-15	Viajes	Caja Negra — PE	Crear viaje exitoso (todos los campos completos)	Ruta con id_ruta=1 , Bus con id_bus=1 , Chofer con id_chofer=1 existen	{ "idRuta": 1, "idBus": 1, "idChofer": 1, "fechaHoraSalida": "

ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	
CN-16	Viajes	Caja Negra — PE	Crear viaje con estado inválido (partición inválida)	—	{ "idRuta": 1, "idBus": 1, "fechaHoraSalida": "2026-08-15T06:00:00" }
CN-17	Viajes	Caja Negra — VLC	Crear viaje con fechaHoraLlegada anterior a fechaHoraSalida	—	{ "idRuta": 1, "idBus": 1, "fechaHoraSalida": "2026-08-15T10:00:00" }
CN-18	Viajes	Caja Negra — PE	Obtener viaje por ID inexistente	No existe viaje con id_viaje=99999	GET /admin/viajes/99999 con token AdminOrSuper

1.4 Módulo Boletos — Creación (POST /admin/viajes/boletos)

ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	
CN-19	Boletos	Caja Negra — PE	Crear boleto exitoso con todos los campos	Viaje id_viaje=1 existe, Pasajero id_pasajero=1 existe, Asiento id_asiento=10 está libre	{ "idViaje": 1, "idUsuario": null, "idPasajero": 1, "idAsiento": 10 }
CN-20	Boletos	Caja Negra — VLC	Precio final con valor límite mínimo (0.01)	Viaje y Pasajero existen	{ ... "precioFinal": 0.01, "canal": "app_bustoke", "codigoQr": "BKT-DUP-001" }
CN-21	Boletos	Caja Negra — VLC	Precio final con valor cero (fuera de límite)	Viaje y Pasajero existen	{ ... "precioFinal": 0.00, ... }
CN-22	Boletos	Caja Negra — PE	Crear boleto con canal inválido	—	{ ... "canal": "whatsapp", ... }
CN-23	Boletos	Caja Negra — PE	Crear boleto con código QR duplicado	Ya existe boleto con codigo_qr = "BKT-DUP-001"	{ ... "codigoQr": "BKT-DUP-001", ... }

1.5 Módulo Público — API Key (/api/v1)

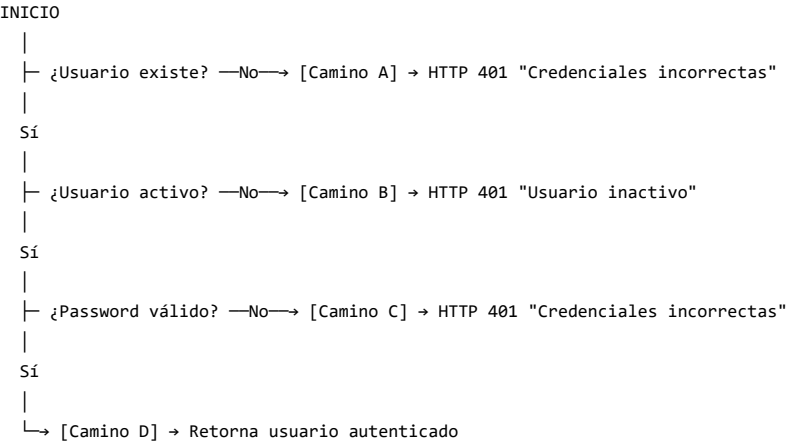
ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	
CN-24	Público	Caja Negra — PE	Consultar asientos disponibles con API Key válida	API Key activa existe en tabla api_keys , viaje id_viaje=1 existe	GET /api/v1/viajes/1/asientos con header x-api-key: <api_key_vali
CN-25	Público	Caja Negra — PE	Consultar asientos sin API Key	—	GET /api/v1/viajes/1/asientos sin header x-api-key
CN-26	Público	Caja Negra — PE	Consultar asientos con API Key inválida	—	GET /api/v1/viajes/1/asientos con header x-api-key: clave-falsa
CN-27	Público	Caja Negra — PE	Crear boleto externo con asiento ya ocupado	Viaje id=1 , asiento id=5 ya tiene un boleto activo	{ "idViaje": 1, "idAsiento": 5, "emailContacto": "c@x.com", "precio

2. Pruebas de Caja Blanca

Componente crítico seleccionado: Función `authenticate_user` en `app/modules/auth/service.py:17-26` y función `list_viajes` en `app/modules/viajes/router.py:24-44` (router con lógica de dispatch condicional).

2.1 Cobertura de Caminos — `authenticate_user`

Flujo con 4 decisiones secuenciales (caminos = 2^3 = 8 combinaciones, 4 terminales):



ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	Datos de Entrada
CB-01	Auth — <code>authenticate_user</code>	Caja Blanca — Cobertura de Caminos	Camino A: usuario no existe en BD (primera	No hay usuario con ese email	{ "email": "ghost@test.com", "password": "x" }

ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	Datos de Entrada
			decisión = False)		
CB-02	Auth — authenticate_user	Caja Blanca — Cobertura de Caminos	Camino B: usuario existe pero inactivo (segunda decisión = False)	Usuario con activo = false en BD	{ "email": "baja@test.com", "password": "x" }
CB-03	Auth — authenticate_user	Caja Blanca — Cobertura de Caminos	Camino C: usuario activo pero password incorrecto (tercera decisión = False)	Usuario activo existe, password_hash es bcrypt válido	{ "email": "activo@test.com", "password": "ClaveErronea999" }
CB-04	Auth — authenticate_user	Caja Blanca — Cobertura de Caminos	Camino D: todas las condiciones verdaderas (flujo feliz)	Usuario activo existe, password correcto	{ "email": "valido@test.com", "password": "MiPassCorrecto1" }

2.2 Cobertura de Decisiones — list_viajes (Router dispatch)

ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	Datos de Entrada	Pasos de
CB-05	Viajes — Router list_viajes	Caja Blanca — Cobertura de Decisiones	Rama id_bus is not None : filtrar por bus	Bus id_bus=2 tiene viajes asignados	GET /admin/viajes?id_bus=2 con token AdminOrSuper	1. Router evalúa p... if id_bus is not get_viajes_by_bus
CB-06	Viajes — Router list_viajes	Caja Blanca — Cobertura de Decisiones	Rama id_ruta is not None : filtrar por ruta	Ruta id_ruta=3 tiene viajes	GET /admin/viajes?id_ruta=3 con token AdminOrSuper	1. Router: id_bus no es None. 2. Llam get_viajes_by_rut
CB-07	Viajes — Router list_viajes	Caja Blanca — Cobertura de Decisiones	Rama admin_agencia forzando id_agencia desde JWT	Token JWT con rol: "admin_agencia" , id_agencia: 5	GET /admin/viajes sin query param id_agencia	1. Router: id_bus None. 2. user_rol → id_agencia = 5 get_viajes_by_age
CB-08	Viajes — Router list_viajes	Caja Blanca — Cobertura de Decisiones	Rama final: get_all_viajes (superadmin sin filtros)	Token JWT con rol: "superadmin" , sin id_bus , id_ruta , id_agencia	GET /admin/viajes con token superadmin	1. Router: todos los falsos. 2. Llama a get_all_viajes(db

3. Pruebas Basadas en Riesgos

Riesgo Crítico 1: Doble venta del mismo asiento en un viaje

Impacto: Pérdida económica directa por overbooking (dos pasajeros con el mismo asiento), insatisfacción del cliente, penalidades.

ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	
RB-01	Boletos — create_boleto	Basada en Riesgo	Solicitud concurrente de 2 compras para el mismo asiento en el mismo viaje	Viaje id=1 existe, Asiento id=10 está libre. Se envían 2 requests POST /admin/viajes/boletos simultáneos (race condition).	Ambos requests: { "idViaje": 1, "idPasajero": 1, "idAsiento": 10, "email": "test@test.com" } y otro con codigoQr: "BKT-RACE-B" y idPasajero: 2
RB-02	Boletos — create_boleto_externo (público)	Basada en Riesgo	Venta de asiento liberado tras pago fallido	Asiento id=5 ocupado por boleto con estado: "cancelado" (antes activo, luego cancelado).	{ "idViaje": 1, "idAsiento": 5, ... }

Riesgo Crítico 2: Acceso no autorizado a datos entre agencias

Impacto: Una agencia puede ver/modificar datos de otra agencia (violación de confidencialidad, regulación de datos personales Ley 29733 Perú).

ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	Datos de Entrada	Pasos de Ejecución
RB-03	Viajes — Listar	Basada en Riesgo	Usuario admin_agencia de agencia 1 intenta acceder a viajes de agencia 2	Token JWT con rol: "admin_agencia", id_agencia: 1. Existen viajes para agencia 2 (rutas con id_agencia=2).	GET /admin/viajes?id_agencia=2 con token de agencia 1	1. Router: detecta admin_agencia=1. 2. Sobrepasa id_agencia con 1 (línea a get_viajes_by_agencia)
RB-04	Finanzas — Liquidaciones	Basada en Riesgo	Usuario admin_agencia lista liquidaciones de otra agencia manipulando parámetros	Token JWT con rol: "admin_agencia", id_agencia: 1	GET /admin/finanzas/liquidaciones sin id_agencia o con id_agencia=2	1. Depende de implementación finanzas/router.py. Si id_agencia desde JWT de agencia 1 puede ver datos de otra. 2.

ID	Componente/Módulo	Tipo de Prueba	Descripción del Caso	Precondiciones	Datos de Entrada	Pasos de Ejecución

Resumen de Hallazgos

Ítem	Tipo	Severidad	Descripción
CN-17	Defecto lógico	Media	No se valida que <code>fechaHoraLlegada > fechaHoraSalida</code> al crear viaje
CN-23	Defecto funcional	Alta	<code>codigo_qr</code> unique constraint no manejada con <code>try-except</code> , causa <code>500</code> en lugar de <code>409 Conflict</code>
CN-21	Defecto ausente	Baja	No hay validación de <code>precioFinal > 0</code>
RB-01	Riesgo arquitectónico	Crítica	Race condition en venta de asientos: 2 requests concurrentes pueden vender el mismo asiento. Sin <code>FOR UPDATE</code> ni unique constraint compuesto.
RB-02	Riesgo mit. parcial	Media	Cancelación libera asiento correctamente gracias a filtro por <code>estado == "activo"</code>
RB-03	Control existente	—	Router de viajes protege correctamente el filtro por agencia.
RB-04	Riesgo pendiente de verificar	Alta	Verificar que otros módulos (finanzas, boletos, pasajeros, rutas) apliquen el mismo filtro de <code>id_agencia</code> desde JWT para <code>admin_agencia</code> .

Documento generado a partir del análisis estático del código fuente.

Repositorio: `bustoke-admin-backend` — FastAPI + async SQLAlchemy 2.x + Pydantic v2 + PostgreSQL 15